

Proje 2, Proje 1 & Proje 6

Rabia Şevval Yaşar
22290015

Proje 2

Github: <https://github.com/Rabia-23/BLM4522>

Video: <https://drive.google.com/file/d/1tTe-ut1kaFR6U8i4JEYbPskB2oS5J9Ui/view?usp=sharing>

Proje 1

Github: <https://github.com/Rabia-23/BLM4522>

Video: https://drive.google.com/file/d/1m02AKCySgy3Yu20qvDSRQC_74e0qKn0C/view?usp=sharing

Proje 6

Github: <https://github.com/Rabia-23/BLM4522>

Video: <https://drive.google.com/file/d/1YD5sUpVGvVje3tmoqkwIsG07l23PXTys/view?usp=sharing>

PROJE 2 - VERİTABANI YEDEKLEME VE FELAKETTEN KURTARMA

1. GİRİŞ

Projede Northwind veritabanı üzerinde tam yedekleme, fark yedekleme ve felaketten kurtarma süreci uygulanmıştır.

Veritabanı yedekleme, üretim sistemlerinin en kritik bakım görevlerinden biridir. Yanlışlıkla silinen veriler, sistem arızaları veya siber saldırılar sonucunda veri kaybı yaşanabilir. Düzgün bir yedekleme stratejisi bu durumlarda veriyi geri kazandırmak için hayati önem taşır.

Araçlar: PostgreSQL 14, Northwind Database, psql, pgAdmin

1.1. YEDEKLEME TÜRLERİ

Tam Yedek (Full Backup)

Veritabanının o anki eksiksiz kopyasıdır. En kapsamlı yedek türüdür ancak en fazla yer kaplar.

Fark Yedek (Differential Backup)

Son tam yedekten bu yana değişen verileri içerir. Tam yedeğe göre daha küçük ve hızlıdır.

Artık Yedek (Incremental Backup)

Yalnızca son yedekten bu yana değişen verileri içerir. En az yer kaplar ancak geri yükleme en karmaşık olan tür.

2. VERİTABANI TANITIMI

Northwind, bir ticaret şirketinin müşteri, sipariş ve ürün verilerini içeren açık kaynaklı bir örnek veritabanıdır. 14 tablo içermektedir.

2.1. YEDEKLEME ÖNCESİ KAYIT SAYILARI

```
SELECT 'customers' AS tablo, COUNT(*) AS kayit FROM customers
UNION ALL SELECT 'orders', COUNT(*) FROM orders
UNION ALL SELECT 'order_details', COUNT(*) FROM order_details
UNION ALL SELECT 'products', COUNT(*) FROM products
UNION ALL SELECT 'employees', COUNT(*) FROM employees;
```

Tablo	Kayıt Sayısı
customers	91
orders	830
order_details	2155
products	77
employees	9

Bu sayılar yedekleme öncesi referans değerler olarak belgelenmiştir.

3. TAM YEDEKLEME

Tam yedek veritabanının o anki eksiksiz kopyasıdır. pgAdmin arayüzü üzerinden alınmıştır.

3.1. PGADMIN İLE TAM YEDEK ALMA

- northwind veritabanına sağ tıkla → Backup...
- Filename: northwind_tam_yedek.backup
- Format: Custom (sıkıştırma ve seçimli yedekleme destekler)
- Backup butonuna tıkla

Custom format seçilmesinin nedeni: sıkıştırma yaparak daha az yer kaplaması ve pgAdmin ile kolayca geri yüklenebilmesidir.

4. YENİ VERİ EKLEME VE FARK YEDEKLEME

Tam yedekten sonra yeni veriler eklenmiştir. Gerçek hayatta veritabanı sürekli değişir — her gün yeni siparişler ve müşteriler gelir.

4.1. YENİ VERİ EKLEME

```
-- Yeni musteriler
INSERT INTO customers (customer_id, company_name, contact_name, country)
VALUES ('YENI1', 'Yeni Sirket A', 'Ahmet Yilmaz', 'Turkey');

INSERT INTO customers (customer_id, company_name, contact_name, country)
VALUES ('YENI2', 'Yeni Sirket B', 'Fatma Kaya', 'Turkey');

-- Yeni siparis
INSERT INTO orders (order_id, customer_id, employee_id,
                    order_date, ship_country)
VALUES (999, 'YENI1', 1, CURRENT_DATE, 'Turkey');
```

4.2. FARK YEDEK ALMA

- northwind veritabanına sağ tıkla → Backup...
- Filename: northwind_fark_yedek.backup
- Format: Custom
- Backup butonuna tıkla

Yedek	Dosya Adı	İçerik
Tam Yedek	northwind_tam_yedek.backup	Orijinal 91 müşteri, 830 sipariş
Fark Yedek	northwind_fark_yedek.backup	Tam yedek + YENI1, YENI2, sipariş 999

5. FELAKET SENARYOSU

Gerçek hayatta yanlış bir DELETE komutu veya sistem arızası kritik verileri silebilir. Bu senaryo bunu simüle etmektedir.

5.1. KRİTİK TABLOLARI SİLME

```
DROP TABLE order_details CASCADE;
DROP TABLE orders CASCADE;
DROP TABLE customers CASCADE;
```

5.2. VERİ KAYBI DOĞRULAMASI

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;
-- customers, orders, order_details artık listede yok
```

Sonuç: 3 kritik tablo tamamen silindi – veri kaybı yaşandı.

6. FELAKETTEN KURTARMA – GERİ YÜKLEME

Fark yedeği kullanılarak geri yükleme yapılmıştır. Fark yedeği tercih edilmesinin nedeni: tam yedeğe göre daha güncel veri içermesidir.

6.1. PGADMIN İLE GERİ YÜKLEME

- northwind veritabanına sağ tıkla → Restore...
- Format: Custom or tar
- Filename: northwind_fark_yedek.backup dosyasını seç
- Restore options → Clean before restore: Açık
- Restore butonuna tıkla

'Clean before restore' seçeneğinin açılmasının nedeni: geri yükleme sırasında mevcut nesnelerle çakışmaları önlemektir.

7. GERİ YÜKLEME DOĞRULAMASI

7.1. TABLO KONTROLÜ

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;
```

Sonuç: 14 tablonun tamamı geri yüklendi.

7.2. KAYIT SAYISI KARŞILAŞTIRMASI

Tablo	Yedek Öncesi	Geri Yüklemeden Sonra	Eşleşme
customers	91 + 2 yeni = 93	93	OK
orders	830 + 1 yeni = 831	831	OK
order_details	2155	2155	OK
products	77	77	OK
employees	9	9	OK

7.3. YENİ VERİLERİN DOĞRULANMASI

```
SELECT customer_id, company_name, contact_name, country
FROM customers WHERE customer_id IN ('YENI1', 'YENI2');
```

```
SELECT order_id, customer_id, order_date, ship_country
FROM orders WHERE order_id = 999;
```

Sonuç: YENI1, YENI2 müşterileri ve 999 numaralı sipariş geri yüklendi.

Önemli not: Tam yedek kullanılsaydı bu yeni veriler kaybolurdu. Fark yedeği sayesinde en güncel hal kurtarıldı.

8. YEDEKLEME LOG SİSTEMİ

Gerçek sistemlerde yedeklemeler otomatik zamanlayıcılarla alınır ve bir log tablosunda kayıt altına alınır.

```
CREATE TABLE backup_log (
    id SERIAL PRIMARY KEY,
    yedek_turu TEXT,
    dosya_adi TEXT,
    alinma_zamani TIMESTAMPTZ DEFAULT now(),
    aciklama TEXT
);
```

```
INSERT INTO backup_log (yedek_turu, dosya_adi, aciklama)
VALUES ('TAM', 'northwind_tam_yedek.backup',
        'İlk tam yedek - tüm veriler dahil');
```

```
INSERT INTO backup_log (yedek_turu, dosya_adi, aciklama)
VALUES ('FARK', 'northwind_fark_yedek.backup',
        'Fark yedek - YENI1, YENI2 ve 99999 siparisi eklendi');
```

```
SELECT * FROM backup_log;
```

9. SONUÇ VE DEĞERLENDİRME

9.1. UYGULANAN ADIMLAR

Adım	İşlem	Sonuç
1	Tam yedek alma	northwind_tam_yedek.backup oluşturuldu
2	Yeni veri ekleme	2 müşteri, 1 sipariş eklendi
3	Fark yedek alma	northwind_fark_yedek.backup oluşturuldu
4	Felaket senaryosu	3 kritik tablo silindi
5	Geri yükleme	Fark yedekten başarıyla geri yüklendi
6	Doğrulama	Tüm tablolar ve kayıtlar eşleşti
7	Log sistemi	Yedekler kayıt altına alındı

9.2. GENEL DEĞERLENDİRME

- Tam ve fark yedek birlikte kullanıldığında hem güvenlik hem de verimlilik sağlanır.
- Fark yedek, son tam yedekten sonra eklenen verilerin de kurtarılmasını sağlar.
- pgAdmin'in Backup/Restore wizard'ı bu süreci kolaylaştırır ve hata riskini azaltır.
- Yedekleme log tablosu, hangi yedeklerin ne zaman alındığını izlemek için kritik öneme sahiptir.

PROJE 1 -VERİTABANI PERFORMANS OPTİMİZASYONU VE İZLEME

1. GİRİŞ

Projede Northwind veritabanı üzerinde kapsamlı bir performans analizi yapılmış ve optimizasyon teknikleri uygulanmıştır.

Veritabanı performansı, büyük ölçekli sistemlerde kritik bir unsurdur. Yanlış yazılmış bir sorgu veya eksik bir index, milyonlarca satırlık bir veritabanında saniyeler sürmesi gereken işlemleri dakikalara taşıyabilir.

Araçlar: PostgreSQL 14, Northwind Database, psql, pgAdmin, pg_stat, EXPLAIN ANALYZE

1.1. ELE ALINAN KONULAR

1

EXPLAIN ANALYZE ile sorgu planı analizi

2

Index yönetimi: index oluşturma, etkisini ölçme ve gereksiz indexleri silme

3

JOIN sorgusu optimizasyonu

4

pg_stat tabloları ile veritabanı istatistiklerinin izlenmesi

2. MEVCUT DURUM ANALİZİ

Performans optimizasyonuna başlamadan önce veritabanının mevcut durumu belgelenmiştir.

2.1. TABLO BOYUTLARI

Tablo	Toplam Boyut	Açıklama
order_details	192 kB	En büyük tablo
orders	176 kB	İkinci büyük tablo
customers	56 kB	
employees	32 kB	

2.2. MEVCUT INDEXLER

Kurulum sonrasında yalnızca PRIMARY KEY indexleri mevcuttu. Sık sorgulanan sütunlarda (country, order_date, ship_country) hiç index bulunmuyordu.

3. EXPLAIN ANALYZE İLE SORGU ANALİZİ

EXPLAIN ANALYZE komutu PostgreSQL'in bir sorguyu nasıl çalıştırdığını gösterir: hangi yöntemi kullandığı, kaç satır taradığı ve ne kadar sürdüğü.

3.1. INDEX ÖNCESİ SORGU PLANLARI

Müşteri Ülkesine Göre Arama

```
EXPLAIN ANALYZE
SELECT * FROM customers WHERE country = 'Germany';
```

Metrik	Değer
Sorgu Yöntemi	Seq Scan (tüm tablo taraması)
Taranan Satır	91 (tümü)
Filtrelenen Satır	80
Döndürülen Satır	11
Çalışma Süresi	0.064 ms

Sipariş Tarihine Göre Arama

```
EXPLAIN ANALYZE
SELECT * FROM orders
WHERE order_date BETWEEN '1997-01-01' AND '1997-12-31';
```

Metrik	Değer
Sorgu Yöntemi	Seq Scan
Taranan Satır	830 (tümü)
Filtrelenen Satır	422
Döndürülen Satır	408
Çalışma Süresi	0.204 ms

Gönderim Ülkesine Göre Arama

```
EXPLAIN ANALYZE
SELECT * FROM orders WHERE ship_country = 'Germany';
```

Metrik	Değer
Sorgu Yöntemi	Seq Scan
Taranan Satır	830 (tümü)
Filtrelenen Satır	708
Döndürülen Satır	122
Çalışma Süresi	0.220 ms

Tespit: Üç sorguda da *Seq Scan* kullanılmaktadır. PostgreSQL sütünlarda index olmadığı için tablonun tamamını baştan sona taramak zorunda kalmaktadır.

4. INDEX YÖNETİMİ

4.1. PERFORMANS INDEXLERİ OLUŞTURMA

```
CREATE INDEX idx_customers_country ON customers(country);
CREATE INDEX idx_orders_order_date ON orders(order_date);
CREATE INDEX idx_orders_ship_country ON orders(ship_country);
```

4.2. INDEX SONRASI KARŞILAŞTIRMA

Sorgu	Index Öncesi	Index Sonrası	Yöntem Değişimi
customers WHERE country	0.064 ms	0.094 ms	Seq Scan (küçük tablo)
orders WHERE order_date	0.204 ms	0.276 ms	Seq Scan (PostgreSQL tercihi)
orders WHERE ship_country	0.220 ms	0.151 ms	Bitmap Index Scan

Önemli Bulgu: PostgreSQL küçük tablolarda index yerine *Seq Scan*'i tercih edebilir. Bu bir hata değil, PostgreSQL'in maliyet bazlı optimizasyon kararıdır. Tablo büyüdükçe indexlerin etkisi çok daha belirgin olur.

4.3. GEREKSİZ INDEX YÖNETİMİ

Gereksiz indexler disk alanı tüketir ve her INSERT/UPDATE işleminde güncellenmek zorunda kaldığından yazma performansını düşürür.

```
-- Gereksiz indexler olusturuldu (gosterim amaclı)
CREATE INDEX idx_orders_freight ON orders(freight);
CREATE INDEX idx_customers_fax ON customers(fax);
```

Index kullanım istatistikleri sorgulandıktan sonra:

Index	Kullanım Sayısı	Durum
idx_orders_ship_country	2 kez	Aktif — tutuldu
idx_orders_freight	0 kez	Gereksiz — silindi
idx_customers_fax	0 kez	Gereksiz — silindi
idx_customers_country	0 kez	Gereksiz — silindi
idx_orders_order_date	0 kez	Gereksiz — silindi

```
DROP INDEX idx_orders_freight;
DROP INDEX idx_customers_fax;
DROP INDEX idx_customers_country;
DROP INDEX idx_orders_order_date;
```

5. JOIN SORGUSU OPTİMİZASYONU

Gerçek sistemlerde en pahalı işlemler genellikle JOIN'lerdir. İki farklı *JOIN* sorgusu analiz edilmiştir.

5.1. AĞIR JOIN SORGUSU

```
EXPLAIN ANALYZE
SELECT o.order_id, od.product_id,
       od.unit_price, od.quantity,
       od.unit_price * od.quantity AS toplam
FROM orders o
JOIN order_details od ON o.order_id = od.order_id
WHERE o.ship_country = 'Germany'
ORDER BY toplam DESC;
```

Metrik	Değer
JOIN Yöntemi	Hash Join
Index Kullanımı	idx_orders_ship_country devreye girdi
Döndürülen Satır	328
Çalışma Süresi	0.792 ms

idx_orders_ship_country indexi bu ağır sorguda devreye girdi. *Bitmap Index Scan* ile orders tablosundan önce Germany siparişleri çekilerek *JOIN* maliyeti düşürüldü.

6. VERİTABANI İSTATİSTİKLERİNİN İZLENMESİ

PostgreSQL *pg_stat* sistem görüntüleri ile veritabanının genel sağlık durumu izlenmiştir.

6.1. TABLO İSTATİSTİKLERİ

Tablo	Aktif Satır	Silinmiş Satır	Seq Scan	Index Scan
order_details	2155	0	4	2
orders	830	0	13	5
customers	91	0	7	0
products	77	0	4	0

Silinmiş satır yok – *VACUUM* işlemi gerekmiyor. Tablo sağlığı iyi durumda.

6.2. BOYUT ANALİZİ

Tablo	Veri Boyutu	Toplam Boyut
orders	112 kB	192 kB (indexler 80 kB kaplıyor)
order_details	96 kB	192 kB (indexler 96 kB kaplıyor)
customers	16 kB	56 kB (indexler 40 kB kaplıyor)

customers tablosunda indexler verinin 3.5 katı yer kaplıyor. Bu gereksiz index oluşturmanın maliyetini göstermektedir.

7. SONUÇ VE DEĞERLENDİRME

7.1. YAPILAN İŞLEMLER ÖZETİ

İşlem	Sonuç
<i>EXPLAIN ANALYZE</i> analizi	3 sorguda Seq Scan tespit edildi
3 performans indexi eklendi	ship_country indexi JOIN sorgusunda devreye girdi
2 gereksiz index silindi	Disk alanı ve yazma performansı korundu
pg_stat izleme	Tablo sağlığı ve index kullanım oranı belirlendi

7.2. ÖNEMLİ BULGULAR

- PostgreSQL küçük tablolarda maliyet bazlı optimizasyon yaparak index yerine Seq Scan'i tercih edebilir – bu normal bir davranıştır.
- idx_orders_ship_country indexi, 2155 satırlık order_details ile yapılan ağır JOIN sorgusunda belirgin performans artışı sağladı.
- Gereksiz indexler disk alanı tüketmesinin yanı sıra her DML işleminde güncellenmek zorunda kalarak yazma performansını düşürür.
- Silinmiş satır (dead tuple) bulunmaması tabloların sağlıklı olduğunu göstermektedir.

PROJE 6 -VERİTABANI YÜKSELTME VE SÜRÜM YÖNETİMİ

1. GİRİŞ

Bu projede Northwind veritabanı v1.0'dan v2.0'a yükseltilemiş, tüm sürüm değişiklikleri DDL trigger ile otomatik olarak izlenmiş ve geri dönüş planı test edilmiştir.

Veritabanı yükseltmeleri dikkatli planlanmazsa veri kaybı veya uygulama hatalarına yol açabilir. Bu nedenle her yükseltme planının mutlaka bir geri dönüş stratejisi içermesi gerekmektedir.

Araçlar: PostgreSQL 14, Northwind Database, psql, DDL Event Trigger, pg_event_trigger, pg_event_trigger_ddl_commands()

1.1. PROJE AŞAMALARI

1

Yükseltme öncesi v1.0 durumunun belgelenmesi

2

DDL event trigger ile sema degisikliklerinin otomatik izlenmesi

3

v2.0 yükseltme planının uygulanması (4 degisiklik)

4

Yükseltme dogrulaması

5

Geri dönüş planının test edilmesi

2. MEVCUT DURUM BELGELEME (V1.0)

Yükseltme öncesinde v1.0 şeması belgelenmiştir. Bu adım, sonradan nelerin değiştiğini anlayabilmek için kritik önem taşır.

2.1. V1.0 ŞEMA ÖZETİ

Tablo	Sütun	Tip	v2.0'da Ne Olacak?
customers	email yok	—	Eklenecek
customers	created_at yok	—	Eklenecek
products	unit_price	real	numeric(10,2) yapılacak
orders	status yok	—	Eklenecek

3. DDL TRIGGER İLE SÜRÜM TAKİBİ

DDL (Data Definition Language) trigger, *CREATE TABLE*, *ALTER TABLE*, *DROP TABLE* gibi yapısal değişiklikler yapıldığında otomatik tetiklenir. Bu sayede manuel takip gerekmeden tüm değişiklikler kayıt altına alınır.

3.1. LOG TABLOSU

```
CREATE TABLE schema_version_log (  
    id          SERIAL PRIMARY KEY,  
    versiyon    TEXT,  
    degisiklik  TEXT,  
    nesne_adi   TEXT,  
    nesne_turu  TEXT,  
    yapan       TEXT,  
    tarih       TIMESTAMPTZ DEFAULT now()  
);
```

3.2. DDL TRIGGER FONKSİYONU VE EVENT TRIGGER

```
CREATE OR REPLACE FUNCTION log_ddl_changes()
RETURNS event_trigger AS $$
DECLARE
    obj record;
BEGIN
    FOR obj IN SELECT * FROM pg_event_trigger_ddl_commands()
    LOOP
        INSERT INTO schema_version_log
            (versiyon, degisiklik, nesne_adi, nesne_turu, yapan)
        VALUES
            ('v2.0', obj.command_tag,
            obj.object_identity,
            obj.object_type,
            current_user);
    END LOOP;
END;
$$ LANGUAGE plpgsql;

CREATE EVENT TRIGGER ddl_logger
ON ddl_command_end
EXECUTE FUNCTION log_ddl_changes();
```

Doğrulama: SELECT evtname, evtevent, evtenabled FROM pg_event_trigger;

Sonuç: ddl_logger | ddl_command_end | O (O = enabled — aktif)

4. V2.0 YÜKSELTME PLANI

4 değişiklik uygulanmıştır. DDL trigger sayesinde her değişiklik otomatik olarak loglanmıştır.

4.1. DEĞİŞİKLİK 1 – CUSTOMERS TABLOSUNA YENİ SÜTUNLAR

```
ALTER TABLE customers ADD COLUMN email VARCHAR(100);
ALTER TABLE customers ADD COLUMN created_at TIMESTAMPTZ DEFAULT now();
```

Gereksesi: Modern sistemlerde email adresi müşteri iletişiminin temel kanalıdır. created_at kayıt oluşturma zamanını tutar.

4.2. DEĞİŞİKLİK 2 – UNIT_PRICE TİP YÜKSELTMESİ

```
ALTER TABLE products ALTER COLUMN unit_price TYPE NUMERIC(10,2);
```

Gerekçesi: real tipi kayan nokta hatalarına açıktır. numeric(10,2) para birimi için çok daha güvenilir – ondalık hassasiyeti garanti altına alır.

4.3. DEĞİŞİKLİK 3 – ORDERS TABLOSUNA STATUS SÜTUNU

```
ALTER TABLE orders ADD COLUMN status VARCHAR(20) DEFAULT 'pending';
```

Gerekçesi: Sipariş durumu takibi için gerekli. Mevcut 830 sipariş otomatik olarak 'pending' statüsüne alındı.

4.4. DEĞİŞİKLİK 4 – VERSİYON TABLOSU

```
CREATE TABLE db_version (  
    id SERIAL PRIMARY KEY,  
    versiyon TEXT NOT NULL,  
    aciklama TEXT,  
    yukleme_tarihi TIMESTAMPTZ DEFAULT now()  
);  
  
INSERT INTO db_version (versiyon, aciklama)  
VALUES ('v2.0', 'Email ve created_at eklendi,  
        unit_price numeric yapıldı, status eklendi');
```

4.5. DDL LOG SONUÇLARI

Versiyon	Değişiklik	Nesne	Tür
v2.0	ALTER TABLE	public.customers	table
v2.0	ALTER TABLE	public.customers	table
v2.0	ALTER TABLE	public.products	table
v2.0	ALTER TABLE	public.orders	table
v2.0	CREATE TABLE	public.db_version	table
v2.0	CREATE INDEX	public.db_version_pkey	index

Toplam 8 değişiklik DDL trigger tarafından otomatik olarak loglanmıştır.

5. YÜKSELTME DOĞRULAMASI

Kontrol	Beklenen	Sonuç
customers.email sütunu	character varying	OK
customers.created_at sütunu	timestamp with time zone	OK
orders.status sütunu	character varying	OK
products.unit_price tipi	numeric	OK
db_version tablosu	v2.0 kaydı mevcut	OK
DDL log kayıt sayısı	8 değişiklik	OK

6. GERİ DÖNÜŞ PLANI (ROLLBACK TO V1.0)

Her yükseltme planının mutlaka bir geri dönüş stratejisi olmalıdır. Aşağıdaki komutlar v2.0 değişikliklerini tamamen geri alır.

```
ALTER TABLE customers DROP COLUMN email;  
ALTER TABLE customers DROP COLUMN created_at;  
ALTER TABLE products ALTER COLUMN unit_price TYPE real;  
ALTER TABLE orders DROP COLUMN status;  
DROP TABLE db_version;
```

6.1. GERİ DÖNÜŞ DOĞRULAMASI

customers tablosu kontrol edildiğinde 11 orijinal sütun görüldü — email ve created_at başarıyla kaldırıldı.

6.2. TAM LOG GEÇMİŞİ

Versiyon	Değişiklik	Nesne
v2.0	ALTER TABLE (x4)	customers, products, orders
v2.0	CREATE TABLE	db_version
v2.0	CREATE INDEX	db_version_pkey
v1.0	ROLLBACK	northwind (manuel kayıt)

DDL trigger hem yükseltme hem geri dönüş adımlarını eksiksiz belgelemiştir.

7. SONUÇ VE DEĞERLENDİRME

7.1. UYGULANAN ADIMLAR ÖZETİ

Aşama	İşlem	Sonuç
1	v1.0 belgeleme	Başlangıç şeması kayıt altına alındı
2	DDL trigger kurulumu	Otomatik değişiklik loglama aktif
3	v2.0 yükseltme (4 değişiklik)	Tüm değişiklikler başarıyla uygulandı
4	Doğrulama	6 kontrol noktası başarılı
5	Rollback testi	v1.0'a başarıyla dönüldü

7.2. GENEL DEĞERLENDİRME

- DDL event trigger, manuel takip gerektirmeden tüm şema değişikliklerini otomatik olarak belgelemiştir.
- unit_price tipinin real'den numeric'e yükselmesi para birimleri için ondalık hassasiyeti garanti altına alır.
- Geri dönüş planı başarıyla test edilmiştir — herhangi bir sorun anında v1.0'a dönmek mümkündür.
- Her yükseltme planının mutlaka bir rollback stratejisi içermesi gerektiğini bu proje somut olarak göstermiştir.